

Review – Optimization for Machine Learning

1 Preliminaries

Definition 1 (Inner Products). *An inner product on $\mathbb{R}^n$ is a map $\langle \cdot, \cdot \rangle : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ with the following properties:*

1. *Symmetry:* $\langle x, y \rangle = \langle y, x \rangle$ for any $x, y \in \mathbb{R}^n$
2. *Additivity:* $\langle x, y + z \rangle = \langle x, y \rangle + \langle x, z \rangle$ for any $x, y, z \in \mathbb{R}^n$
3. *Homogeneity:* $\langle \lambda x, y \rangle = \lambda \langle x, y \rangle$ for $\lambda \in \mathbb{R}$ and any $x, y \in \mathbb{R}^n$
4. *Positive definiteness:* $\langle x, x \rangle \geq 0$ for any $x \in \mathbb{R}^n$ and $\langle x, x \rangle = 0$ iff $x = 0$

► **Norm** of a vector $\|x\|$ is informally measure of the “length” of the vector. A norm $\|\cdot\|$ on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfying

- For all $x \in \mathbb{R}^n$, $\|x\| \geq 0$ (non-negativity).
- $\|x\| = 0$ if and only if $x = 0$ (definiteness).
- For all $x \in \mathbb{R}^n$, $\lambda \in \mathbb{R}$, $\|\lambda x\| = |\lambda| \|x\|$ (homogeneity).
- For all $x, y \in \mathbb{R}^n$, $\|x + y\| \leq \|x\| + \|y\|$ (triangle inequality).

► Dot product: $\langle x, y \rangle = \sum_{i=1}^n x_i y_i = x^\top y$ for any $x, y \in \mathbb{R}^n$

► The norm associated with the dot-product is the so-called Euclidean norm or ℓ_2 -norm:

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2} \quad \text{for all } x \in \mathbb{R}^n$$

Note that $\|x\|_2^2 = x^\top x$.

► Other examples of norms are the following:

$$\|x\|_1 = \sum_{i=1}^n |x_i| \quad \|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \text{ for a real number } p \geq 1$$

Lemma 1 (Cauchy-Schwartz Inequality). *For any $x, y \in \mathbb{R}^n$: $|x^\top y| \leq \|x\| \|y\|$*

Lemma 2 (Triangle Inequality). *For any $x, y \in \mathbb{R}^n$: $\|x + y\| \leq \|x\| + \|y\|$*

Theorem 1 (Young’s Inequality). *If $a \geq 0$ and $b \geq 0$ are nonnegative real numbers and if $p > 1$ and $q > 1$ are real numbers such that $\frac{1}{p} + \frac{1}{q} = 1$, then $ab \leq \frac{a^p}{p} + \frac{b^q}{q}$.*

- ▶ A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is **positive definite** (PD) if for all non-zero vectors $x \in \mathbb{R}^n$, $x^\top Ax > 0$. This is usually denoted by $A \succ 0$.
- ▶ If $x^\top Ax \geq 0$, we call A a **positive semidefinite** (PSD) matrix, $A \succeq 0$.
- ▶ A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is called **indefinite** if there exist $x, y \in \mathbb{R}^n$ such that $x^\top Ax > 0$ and $y^\top Ay < 0$.
- ▶ If A is positive definite, then $-A$ is negative definite and vice versa.
- ▶ If A is positive semidefinite then $-A$ is negative semidefinite and vice versa.
- ▶ Given an $n \times n$ matrix A , an **eigenvalue** for A is a number λ which, for some nonzero vector v , satisfies $Av = \lambda v \iff (A - \lambda I)v = 0$, where I is the identity matrix and vector v is the **eigenvector** associated with the λ .
- ▶ A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is a positive semidefinite (PSD) if and only if all eigenvalues of A are non-negative. (Similarly, it is PD iff all eigenvalues are positive.)
- ▶ A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is indefinite if and only if it has at least one positive eigenvalue and at least one negative eigenvalue.
- ▶ **Singular Value Decomposition (SVD):** A is $m \times n$ matrix with rank r , then:

$$A = U\Sigma V^\top, \quad \text{where } U \in \mathbb{R}^{m \times m}, \Sigma \in \mathbb{R}^{m \times n}, V \in \mathbb{R}^{n \times n}$$

- $U^\top U = UU^\top = I_m$, $V^\top V = VV^\top = I_n$ and $\Sigma \in \mathbb{R}^{m \times n}$ is a rectangular diagonal matrix with non-negative real numbers on the diagonal (the singular values), typically denoted as:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \text{ where } \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 \text{ and } r = \text{rank}(A).$$

- ▶ A matrix norm $\|\cdot\| : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ is a function satisfying the following properties for all scalar $\lambda \in \mathbb{R}$ and matrices $A, B \in \mathbb{R}^{m \times n}$:
 - $\|A\| \geq 0$ (non-negativity).
 - $\|A\| = 0$ if and only if $A = 0_{m \times n}$ (definiteness).
 - $\|\lambda A\| = |\lambda| \|A\|$ (homogeneity).
 - $\|A + B\| \leq \|A\| + \|B\|$ (triangle inequality).
- ▶ Matrix norms are particularly useful if they are also sub-multiplicative: $\|AB\| \leq \|A\| \|B\|$.
- ▶ If we choose the vector v with the largest growth factor, that gives an important matrix norm $\|A\| : \|A\| = \max_{v \neq 0} \frac{\|Av\|}{\|v\|}$.

$$\|A\|_2 = \sigma_1 \text{ (largest singular value),}$$

$$\|A\|_1 = \text{largest } \ell_1 \text{ - norm of columns of } A, \quad \|A\|_\infty = \text{largest } \ell_1 \text{ - norm of rows of } A$$

- **Nuclear norm** or trace norm: $\|A\|_{nuclear} = \sigma_1 + \dots + \sigma_r$
- **Frobenius norm**: $\|A\|_F = \sigma_1^2 + \dots + \sigma_r^2$
- **Matrix Completion**: Consider the dataset of user-media preference pairs as a partially-observed matrix. Every row represents a person, and every column represents a movie:

$$M_{ij} = \begin{cases} -1, & \text{person } i \text{ dislikes movie } j \\ 1, & \text{person } i \text{ likes movie } j \\ \star, & \text{preference unknown} \end{cases}$$

There are only k factors that determine a persons preference over movies. Define $\|\cdot\|_{OB}$ as the norm only on the observed (non starred) entries of M , i.e., $\|X\|_{OB} = \sum_{M_{ij} \neq \star} X_{ij}^2$. Model the problem as $\min_{rank(X) \leq k} \|X - M\|_{OB}$ or $\min_{X_{ij}=M_{ij}} \|X\|_{nuclear}$.

- **Principal Component Analysis (PCA)** uses the largest σ s connected to the first u s and v s to understand the information in a matrix of data:

$$A_k = \sigma_1 u_1 v_1^\top + \dots + \sigma_k u_k v_k^\top, \quad \text{s.t. } rank(A) = k$$

A_k solves an optimization problem: $\min_{A_k} \|A - A_k\|$ such that $rank(A_k) \leq k$.

- The **gradient** of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at point x is a vector valued function $\nabla f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ whose components are the partial derivatives of f at point x .
 - Gradient of linear function $f(x) = a^\top x + b$ is $\nabla f(x) = a$, where $a, x \in \mathbb{R}^n$ and $b \in \mathbb{R}$.
 - $f(x) = x^\top A x \implies \nabla f(x) = (A^\top + A)x$, for $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$.
- The **Hessian** of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with respect to x is a $n \times n$ matrix of partial derivatives:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \frac{\partial^2 f(x)}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{bmatrix}.$$

- **Taylor Expansion.** A first order approximation uses the first derivative of a function f evaluated at x to approximate the value of f at a neighborhood of x :

$$f(y) \approx f(x) + \nabla f(x)^\top (y - x).$$

- Similarly, a second order approximation uses the first and second derivatives of a function f evaluated at x to approximate the value of f at a neighborhood of x :

$$f(y) \approx f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2} (y - x)^\top \nabla^2 f(x) (y - x).$$

Definition 2 (Big O Notation). Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}$ be two real valued functions. We denote $f(x) = \mathcal{O}(g(x))$, as $x \rightarrow \infty$ if there exist $\alpha, x_0 > 0$ such that $|f(x)| \leq \alpha |g(x)|$ for all $x > x_0$. For example, $5x^4 + 2x + 3 = \mathcal{O}(x^4)$.

2 Convex Optimization

Definition 3 (Convex Function). A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function if it satisfies

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \quad \forall \alpha \in (0, 1), x, y \in \mathbb{R}^n$$

Moreover, f is called strictly convex if the inequality is strict for any $x \neq y$.

- f is **convex** if and only if

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \quad \text{for all } x, y$$

- f is **strictly convex** if and only if

$$f(y) > f(x) + \nabla f(x)^\top (y - x) \quad \text{for all } x, y$$

- f is μ -**strongly convex** if and only if there exists $\mu > 0$:

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|x - y\|^2 \quad \text{for all } x, y$$

- If f is twice continuously differentiable, then

$$\begin{aligned} f \text{ is } \mathbf{convex} &\iff \nabla^2 f(x) \succeq 0 \text{ for all } x \\ f \text{ is } \mathbf{strictly convex} &\iff \nabla^2 f(x) \succ 0 \text{ for all } x \\ f \text{ is } \mathbf{strongly convex} &\iff \nabla^2 f(x) \succeq \mu I \text{ for all } x \end{aligned}$$

- If the objective function is **convex**, then any local minimum is a global minimum
- If the objective function is **strictly convex**, then the minimum is unique
- If the objective function is **strongly convex**, then the minimum is unique

Definition 4 (Convex Set). A set S is convex if for any $x, y \in S$, $\alpha x + (1 - \alpha)y \in S$ for all $\alpha \in [0, 1]$.

Definition 5 (Convex Optimization).

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & x \in X \end{aligned}$$

is convex if f is a convex function and X is a convex set. A **functional form** of a convex problem can be written as

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

where $f, g_1, \dots, g_m : \mathbb{R}^n \rightarrow \mathbb{R}$ are convex functions and $h_1, h_2, \dots, h_p : \mathbb{R}^n \rightarrow \mathbb{R}$ are affine functions.

3 Optimality Condition

- An unconstrained optimization problem is of the form

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in \mathbb{R}^n, \end{aligned} \quad (\text{Opt})$$

where $x \in \mathbb{R}^n$ is the **variable** and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the **objective function**.

Name	Assumptions	Conclusion
First-order necessary condition	$f : \mathbb{R}^n \rightarrow \mathbb{R}$, x^* is a local minimum, partial derivatives of f exist at x^*	$\nabla f(x^*) = 0$
Second-order necessary condition	$f : \mathbb{R}^n \rightarrow \mathbb{R}$, x^* is a local minimum, f is twice continuously differentiable	$\nabla^2 f(x^*) \succeq 0$
Sufficient optimality condition	$f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable, $\nabla f(x^*) = 0$, $\nabla^2 f(x^*) \succ 0$	x^* is a strict local minimum
Stationary point	$f : \mathbb{R}^n \rightarrow \mathbb{R}$, partial derivatives defined at x^*	x^* is stationary if $\nabla f(x^*) = 0$
Saddle point	$f : \mathbb{R}^n \rightarrow \mathbb{R}$ continuously differentiable, x^* is a stationary point	If x^* is neither a local minimum nor maximum, then it's a saddle point
Sufficient condition for saddle point	$f : \mathbb{R}^n \rightarrow \mathbb{R}$ twice continuously differentiable, x^* is a stationary point, $\nabla^2 f(x^*)$ is indefinite	x^* is a saddle point
Global optimality condition	$f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable, $\nabla^2 f(x) \succeq 0 \forall x$, and x^* is stationary	x^* is a global minimum

- Consider the following optimization problem:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in C \end{aligned} \quad (\text{P})$$

Definition 6 (Stationary Point for Constrained Optimization). *Let f be a continuously differentiable function over a closed and convex set C . Then x^* is called a **stationary point** of (P) if*

$$\nabla f(x^*)^\top (x - x^*) \geq 0, \text{ for all } x \in C$$

Theorem 2. *Let f be a continuously differentiable function over a nonempty closed convex set C , and let x^* be a local minimum of (P). Then x^* is a stationary point of (P).*

- For convex problems, stationarity is a necessary and sufficient condition

Theorem 3. *Let f be a continuously differentiable convex function over a nonempty closed and convex set $C \subseteq \mathbb{R}^n$. Then x^* is a stationary point of (P) iff x^* is an optimal solution of (P).*

4 Gradient-based Methods

- Consider the following convex optimization problem:

$$\min_x f(x)$$

- f has a **Lipschitz continuous gradient** if there exists $L > 0$ such that

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in \mathbb{R}^n$$

- If function f is twice differentiable such that $\|\nabla^2 f(x)\| \leq L$ for all x and some $L > 0$, then f has a Lipschitz continuous gradient with constant L .
- If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a differentiable convex function with L -Lipschitz continuous gradient, then the following conditions are equivalent for all x, y :

- (i) $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$
- (ii) $f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|x - y\|^2$
- (iii) $\langle \nabla f(x) - \nabla f(y), x - y \rangle \leq L\|x - y\|^2$
- (iv) $\|\nabla f(x) - \nabla f(y)\|^2 \leq L(\nabla f(x) - \nabla f(y))^\top (x - y)$
- (v) $\|\nabla f(x) - \nabla f(y)\|^2 \leq 2L(f(x) - f(y) - \nabla f(x)^\top (x - y))$

Algorithm 1 Gradient Method

Initialization: pick $x_0 \in \mathbb{R}^n$ arbitrarily
for $k = 0, 1, 2, \dots$ **do**
 select $\alpha < 2/L$
 set $x_{k+1} = x_k - \alpha \nabla f(x_k)$
 if $\|\nabla f(x_{k+1})\| \leq \epsilon$ then STOP and x_{k+1} is the output
end for

Definition 7 (Projection). *Given a nonempty closed convex set C , the orthogonal projection operator $P_C : \mathbb{R}^n \rightarrow C$ is defined by $P_C(x) = \operatorname{argmin}\{\|y - x\|^2 : y \in C\}$.*

set (C)	$P_C(x)$	Assumptions
$\mathbb{R}_+^n$	$\max(x, 0)$	–
$\text{Box}(\ell, u)$	$\min\{\max\{x, \ell\}, u\}$	$\ell \leq u$
$\text{B}(0, r)$	$\frac{x}{\ x\ }r$	$r > 0$
$\{x \mid Ax = b\}$	$x - A^\top (AA^\top)^{-1}(Ax - b)$	$A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, A$ full row rank
$\{x \mid a^\top x \leq b\}$	$x - \frac{\max\{a^\top x - b, 0\}}{\ a\ ^2}a$	$0 \neq a \in \mathbb{R}^n, b \in \mathbb{R}$

Table 2: Orthogonal Projections

Algorithm 2 Projected Gradient Method

Initialization: pick $x_0 \in \mathbb{R}^n$ arbitrarily
for $k = 0, 1, 2, \dots$ **do**
 select $\alpha = 1/L$
 set $x_{k+1} = P_C(x_k - \alpha_k \nabla f(x_k))$
 if $\|x_{k+1} - x_k\| \leq \epsilon$ then STOP and x_{k+1} is the output
end for

- Consider optimization problem $\min_x f(x)$ where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex and possibly **non-differentiable**

Definition 8 (Subgradient). g is a subgradient of a convex function f at x if

$$f(y) \geq f(x) + g^\top(y - x), \quad \forall y$$

Definition 9 (Subdifferential). The subdifferential of a convex function f at $x \in \text{dom}(f)$ is the collection of all subgradients of f at x : $\partial f(x) = \{g : f(y) \geq f(x) + g^\top(y - x)\}$.

Projected Subgradient Method: Consider $\min_{x \in C} f(x)$ where C is a closed convex subset of $\mathbb{R}^n$:

$$x_{k+1} = P_C(x_k - \alpha_k g_k).$$

where $g_k \in \partial f(x_k)$, any subgradient of f at x_k . For convergence, subgradient method requires the function f to be Lipschitz.

- Consider the following convex optimization problem:

$$\min f(x) \triangleq g(x) + h(x),$$

where g is a convex and differentiable function and h is convex and possibly non-differentiable, but simple, e.g., $h(x) = \|x\|_1$.

Definition 10 (Prox). prox is a function of h and α , and is referred to as the proximal map of h :

$$\text{prox}_{h,\alpha}(x) = \underset{z}{\operatorname{argmin}} \frac{1}{2\alpha} \|z - x\|^2 + h(z).$$

$h(x)$	$\text{prox}_{h,\alpha}(x)$	Assumptions
$\lambda\ x\ $	$\left(1 - \frac{\alpha\lambda}{\max\{\ x\ , \alpha\lambda\}}\right)x$	$\lambda > 0$
$\lambda\ x\ ^3$	$\frac{2}{1 + \sqrt{1 + 12\alpha\lambda\ x\ }}x$	$\lambda > 0$
$\lambda\ x\ _1$	$[x - \alpha\lambda e]_+ \odot \operatorname{sgn}(x)$	$\lambda > 0$

Table 3: Prox Computation

Proximal Gradient Descent: Choose initial x_0 and then repeat:

$$x_{k+1} = \text{prox}_{h,\alpha_k}(x_k - \alpha_k \nabla g(x_k)).$$

- Consider the following convex optimization problem:

$$\min_x f(x)$$

Momentum gradient descent, or the **heavy-ball algorithm**: Introduces a *momentum* term $\beta(x_k - x_{k-1})$, where β is a hyperparameter

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}).$$

Algorithm 3 Nesterov's Accelerated Gradient Method

Initialization: pick $x_0 \in \mathbb{R}^n$ arbitrarily, let $x_{-1} = x_0$

for $k = 0, 1, 2, \dots$ **do**

$$y = x_k + \frac{k-2}{k+1}(x_k - x_{k-1})$$

$$x_{k+1} = y - \alpha \nabla f(y)$$

end for

- Consider the following convex optimization problem:

$$\min f(x) \triangleq g(x) + h(x),$$

where g is a convex and differentiable function and h is convex and possibly non-differentiable, but simple, e.g., $h(x) = \|x\|_1$.

Algorithm 4 Accelerated Proximal Gradient Method – FISTA

Initialization: pick $x_0 \in \mathbb{R}^n$ arbitrarily, $t_0 = 1$, $y_0 = x_0$

for $k = 0, 1, 2, \dots$ **do**

$$x_{k+1} = \text{prox}_{h,\alpha}(y_k - \alpha \nabla g(y_k))$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$$

$$y_{k+1} = x_{k+1} + \left(\frac{t_k - 1}{t_{k+1}} \right) (x_{k+1} - x_k)$$

end for

Method	Convex function	Strongly convex function
Gradient Descent (GD)	$\mathcal{O}(1/\epsilon)$	$\mathcal{O}(\log(1/\epsilon))$
Accelerated GD	$\mathcal{O}(1/\sqrt{\epsilon})$	$\mathcal{O}(\log(1/\epsilon))$
Proximal GD	$\mathcal{O}(1/\epsilon)$	$\mathcal{O}(\log(1/\epsilon))$
Subgradient	$\mathcal{O}(1/\epsilon^2)$	$\mathcal{O}(1/\epsilon)$

Table 4: Convergence Complexity

5 Stochastic Optimization

- Consider the following problem of minimizing an average of convex functions:

$$\min_x f(x) \triangleq \frac{1}{n} \sum_{i=1}^n f_i(x)$$

- Gradient descent will be very costly when $n \gg 1$. Instead, we can apply the following algorithm:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k)$$

where $i_k \in \{1, \dots, n\}$ is some chosen index at iteration k .

Algorithm 5 Mini-batch Stochastic Gradient Descent (SGD)

Initialization: pick $x_0 \in \mathbb{R}^m$ arbitrarily, choose $b \ll n$

for $k = 0, 1, 2, \dots, T - 1$ **do**

choose subset $I_k \subset \{1, \dots, n\}$, where $|I_k| = b \ll n$, uniformly at random

choose $\alpha_k = \min\{\frac{1}{2L}, \frac{1}{\sqrt{T}}\}$ or $\alpha_k = 1/\sqrt{k}$

set $x_{k+1} = x_k - \frac{\alpha_k}{b} \sum_{i \in I_k} \nabla f_i(x_k)$

end for

Jensen's inequality: Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function and X a random variable with finite expectation. Then, $f(\mathbb{E}[X]) \leq \mathbb{E}[f(x)]$.

Stochastic Average Gradient (SAG):

- Maintain table, containing $g^{(i)}(x) = \nabla f_i(x)$, $i = 1, \dots, n$
- Initialize x_0 and set $g = 0$ and $g^{(i)} = 0$ for all $i = 1, \dots, n$
- At step $k = 1, 2, 3, \dots$ pick random $i_k \in \{1, \dots, n\}$ and then let

$$g_k^{(i_k)} = \nabla f_{i_k}(x_{k-1})$$

set all other $g_k^{(i_k)} = g_{k-1}^{(i_k)}$, $i \neq i_k$, i.e., they stay the same

- Set $\alpha_k = 1/(16L)$ and update

$$x_k = x_{k-1} - \alpha_k \left(\frac{1}{n} \left(g_k^{(i_k)} - g_{k-1}^{(i_k)} \right) + \frac{1}{n} \sum_{i=1}^n g_{k-1}^{(i)} \right)$$

SAGA: The update of SAGA is $x_k = x_{k-1} - \alpha_k \left(g_k^{(i_k)} - g_{k-1}^{(i_k)} + \frac{1}{n} \sum_{i=1}^n g_{k-1}^{(i)} \right)$ for $\alpha_k = 1/(3L)$.

- SAGA has a higher variance than SAG but is unbiased.
-

Algorithm 6 SVRG**Parameters:** update frequency m and stepsize α **Initialization:** pick $\tilde{x}_0 \in \mathbb{R}^n$ arbitrarily**for** $s = 1, 2, \dots$ **do**

$$\tilde{x} = \tilde{x}_{s-1}$$

$$\tilde{g} = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$$

$$x_0 = \tilde{x}$$
for $t = 1, 2, \dots, m$ **do**

Randomly pick $i \in \{1, \dots, n\}$ and update

$$x_t = x_{t-1} - \alpha (\nabla f_i(x_{t-1}) - \nabla f_i(\tilde{x}) + \tilde{g})$$
end for**Option I:** $\tilde{x}_s = x_m$ **Option I:** $\tilde{x}_s = x_t$ for randomly chosen $t \in \{1, \dots, m\}$ **end for**

- Consider $\min_x F(x) \triangleq \frac{1}{n} \sum_{i=1}^n f_i(x) + h(x)$, where f_i s are convex and differentiable, h is convex but possibly nonsmooth

Algorithm 7 SVRG++**Initialization:** pick $\tilde{x}_0 \in \mathbb{R}^n$ arbitrarily**for** $s = 1, 2, \dots, S$ **do**

$$\tilde{x} = \tilde{x}_{s-1}$$

$$\tilde{g} = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$$

$$m_s = 2^s m_0$$
for $t = 1, 2, \dots, m_s$ **do**

Randomly pick $i \in \{1, \dots, n\}$ and update

$$g = \nabla f_i(x_{t-1}) - \nabla f_i(\tilde{x}) + \tilde{g}$$

$$x_t = \text{prox}_{h, \alpha}(x_{t-1} - \alpha g)$$
end for $\tilde{x}_s = \frac{1}{m_s} \sum_{t=1}^{m_s} x_t$ $x_0 = x_{m_s}$ **end for**

- For convergence, SGD requires conditional boundedness of the variance assumption, $\mathbb{E}[\|w_k\|^2 \mid x_k] \leq \sigma^2$, where $w_k \triangleq \nabla f(x_k) - \nabla f_{i_k}(x_k)$. However, SAG/SAGA and SVRG do not require this assumption.

Method	Convex function	Strongly convex function
Gradient Descent (GD)	$\mathcal{O}(1/\epsilon)$	$\mathcal{O}(\log(1/\epsilon))$
SGD	$\mathcal{O}(1/\epsilon^2)$	$\mathcal{O}(1/\epsilon)$
SAG/SAGA	$\mathcal{O}((n+L)/\epsilon)$	$\mathcal{O}(\log(1/\epsilon))$
SVRG	$\mathcal{O}(n \log(1/\epsilon) + L/\epsilon)$	$\mathcal{O}(\log(1/\epsilon))$

Table 5: Convergence Complexity

6 Convex Problems

1. Least Square. We are given a linear system of the form

$$Ax = b$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. System usually has no solution, so we find an approximate solution by solving the following optimization problem:

$$\min_x f(x) \triangleq \|Ax - b\|^2.$$

Note that $\nabla f(x) = 2A^\top(Ax - b)$ and Lipschitz of gradient is $L = 2\|A\|^2$.

2. Data Fitting. Data: (s_i, t_i) , $i = 1, \dots, m$, where $s_i \in \mathbb{R}^n$ and $t_i \in \mathbb{R}$:

$$t_i = s_i^\top a + b$$

Define $S = \begin{bmatrix} s_1^\top & 1 \\ s_2^\top & 1 \\ \vdots & \vdots \\ s_m^\top & 1 \end{bmatrix}$, $t = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_m \end{bmatrix}$, and $x = \begin{bmatrix} a \\ b \end{bmatrix}$, hence the least squares problem is

$$\min_{x \in \mathbb{R}^{n+1}} \frac{1}{2} \|Sx - t\|^2$$

where $a = x(1:n)$ and $b = x(n+1)$.

3. Dual Support Vector Machine (SVM). Suppose that we are given two types of points in $\mathbb{R}^n$: type A and type B points. Our objective is to find a linear separator, which is a hyperplane of the form

$$H(w, \beta) = \{x \in \mathbb{R}^n : w^\top x + \beta = 0\},$$

for which the type A and type B points are in its opposite sides:

$$\begin{aligned} w^\top x_i + \beta &< 0, & i = 1, 2, \dots, m, \\ w^\top x_i + \beta &> 0, & i = m+1, m+2, \dots, m+p. \end{aligned}$$

Define $X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_{m+p}^\top \end{bmatrix}$, $Y = \text{diag}(y)$ and $Q = YXX^\top Y$, then we have

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{m+p} \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j (x_i x_j^\top) & \max_{\alpha} \quad & e^\top \alpha - \frac{1}{2} \alpha^\top Q \alpha \\ \text{s.t.} \quad & \sum_{i=1}^{m+p} \alpha_i y_i = 0, & \text{s.t.} \quad & \sum_{i=1}^{m+p} \alpha_i y_i = 0, \\ & \alpha \geq 0. & & \alpha \geq 0. \end{aligned}$$

Underlying Assumption: The two sets of points are linearly separable, meaning that the set of inequalities has a solution. If not, the last constraint will be $0 \leq \alpha \leq C$, where C is the tradeoff parameter between error and margin. In this case, we call the problem soft-margin SVM.

4. Lasso. Consider data points (a_i, b_i) , $i = 1, \dots, m$, where $a_i \in \mathbb{R}^n$ and $b_i \in \mathbb{R}$. Suppose $A \in \mathbb{R}^{m \times n}$ denote the predictor matrix (whose i^{th} row is a_i) and b denote the response vector. Lasso problem is defined as:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1,$$

where $\lambda \geq 0$ is tuning parameter. Lasso is useful when problem is sparse and larger values of the tuning parameter λ typically means sparser solutions.

5. Image Deblurring. Consider the following linear inverse problem:

$$Ax = b + w,$$

where $A \in \mathbb{R}^{m \times m}$, $b \in \mathbb{R}^m$ are known, w is an unknown noise (or perturbation) vector, and x is the “true” and unknown signal/image to be estimated. In image blurring problems, for example, $b \in \mathbb{R}^m$ represents the blurred image, and $x \in \mathbb{R}^m$ is the unknown true image. Both b and x are formed by stacking the columns of their corresponding two-dimensional image. The matrix A describes the blur operator. The problem of estimating x from the observed blurred and noisy image b is called an image deblurring problem:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1.$$

6. Group Lasso Logistic Regression. Group lasso allows predefined groups of covariates to jointly be selected into or out of a model. This is useful in many settings, perhaps most obviously when a categorical variable is coded as a collection of binary covariates. In this case, group lasso can ensure that all the variables encoding the categorical covariate are included or excluded together.

Suppose we have features $X \in \mathbb{R}^{n \times (p+1)}$ that we divide into J groups:

$$X = [\mathbf{1}, X_{(1)}, \dots, X_{(J)}],$$

where $\mathbf{1} = (1, \dots, 1) \in \mathbb{R}^n$ and each $\dots, X_{(j)} \in \mathbb{R}^{n \times p_j}$. To achieve sparsity over groups of features, rather than individual features, we can use a *group lasso penalty*. Write $\beta = (\beta_0, \beta_{(1)}, \dots, \beta_{(J)}) \in \mathbb{R}^{p+1}$, where β_0 is an intercept term and each $\beta_{(j)} \in \mathbb{R}^{p_j}$. Consider the problem

$$\min_{\beta} g(\beta) + \lambda \sum_{j=1}^J w_j \|\beta_{(j)}\|_2,$$

where g is a loss function and $\lambda \geq 0$ is a tuning parameter. The penalty $h(\beta) = \lambda \sum_{j=1}^J w_j \|\beta_{(j)}\|_2$ is called the group lasso penalty. A common choice for w_j is $\sqrt{p_j}$ to adjust for the group size.

Let $y \in \{0, 1\}^n$ be a binary label, and let g be the logistic loss

$$g(\beta) = -\frac{1}{n} \sum_{i=1}^n y_i (X\beta)_i + \frac{1}{n} \sum_{i=1}^n \log(1 + \exp\{(X\beta)_i\}).$$

If we define $f(\beta) = \frac{1}{n} \sum_{i=1}^n \log(1 + \exp\{(X\beta)_i\})$, then $\nabla f(\beta) = \frac{1}{n} \left(-X^\top y + X^\top \left(\frac{\exp(X\beta)}{1 + \exp(X\beta)} \right) \right)$ and Lipschitz of the gradient is $L = \frac{1}{4n} \|X\|^2$.